

系统开发工具基础第四周实验报告

姓名：刘静雅

学号：24160001055

日期：2026 年 9 月 14 日

目录

1	实验环境与目录结构	2
2	课上实验	2
2.1	第 13 题建立可执行的本地质量门禁	2
2.2	第 14 题让 Make 只重建真正受影响的产物	4
2.3	第 15 题把本地 API 数据转换为可读报告	7
2.4	第 16 题修复并交付一个陌生的小型工具仓库	9
3	实验总结与感悟	12

1 实验环境与目录结构

本周实验在 WSL 的 Bash 环境中完成，仓库目录为 `~/sys-dev-tools-lab/week04`。实验主要使用 Python、pytest、ruff、Make、curl、jq、Git 等工具，完成代码质量检查、自动化构建、本地 API 数据处理与软件包交付等任务。

本周实验目录结构如下：

```
week04/  
|-- q13/  
|-- q14/  
|-- q15/  
|-- q16/  
|-- report_assets/  
|   |-- practice/  
|-- report.tex
```

2 课上实验

2.1 第 13 题建立可执行的本地质量门禁

题目原文

复用第 10 题完成的 greetlab 包，在 q13 中建立一个轻量本地质量检查。

- 复制 q10 为 q13；在 `pyproject.toml` 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 `ruff format`、`ruff check` 和 `pytest`，修复全部问题，不得全局忽略规则。
- 编写 `check.sh`，依次执行 `ruff format --check`、`ruff check` 和 `pytest`；运行并保证返回 0。

实验步骤

首先复用前一周完成的 greetlab 项目，将 q10 复制为 q13。在 `pyproject.toml` 中补充 pytest 与 ruff 的最小配置，使测试框架能够定位 `src` 与 `tests` 目录，同时为代码格式检查和静态检查提供统一规则。

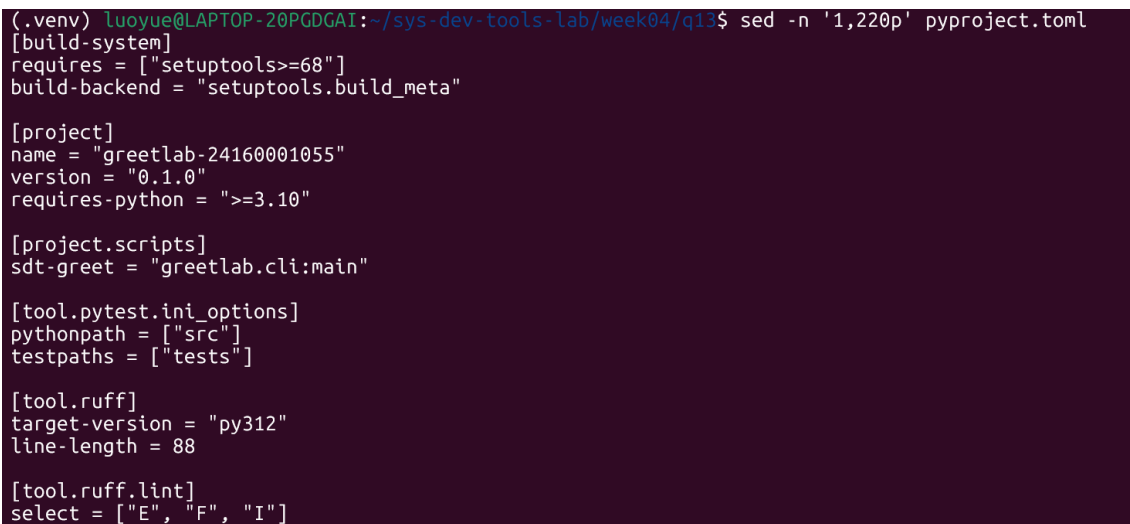
随后补充正常姓名和空白姓名两个测试用例。正常姓名测试验证程序能够输出正确问候语；空白姓名测试验证程序能够拒绝无效输入。完成代码修改后，分别运行 `pytest`、`ruff format --check .` 和 `ruff check`。

最后编写 `check.sh`，将格式检查、静态检查和单元测试串联为一个统一的本地质量门禁入口。脚本的核心形式如下：

```
#!/usr/bin/env bash
set -euo pipefail

ruff format --check .
ruff check .
pytest
```

实验结果与分析

A terminal screenshot showing the contents of a pyproject.toml file. The file is configured with build-system (setuptools), project metadata (name: greetlab-24160001055, version: 0.1.0, requires-python: >=3.10), project scripts (sdt-greet), and tool configurations for pytest (pythonpath, testpaths) and ruff (target-version, line-length, lint rules).

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ sed -n '1,220p' pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-24160001055"
version = "0.1.0"
requires-python = ">=3.10"

[project.scripts]
sdt-greet = "greetlab.cli:main"

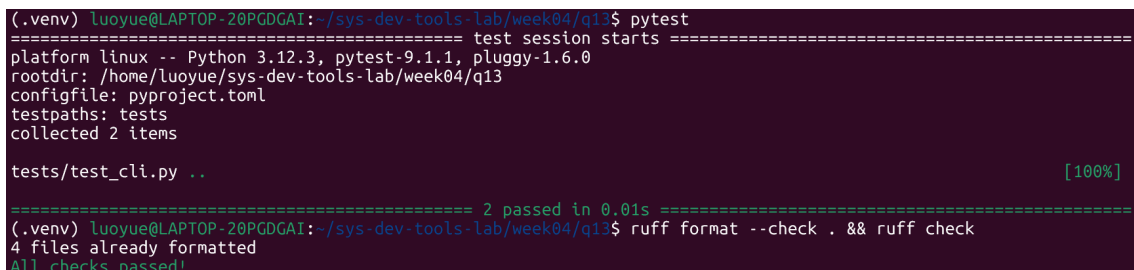
[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.ruff]
target-version = "py312"
line-length = 88

[tool.ruff.lint]
select = ["E", "F", "I"]
```

图 1: 第 13 题 pyproject.toml 中的 pytest 与 ruff 配置

截图显示，项目在 `pyproject.toml` 中配置了 pytest 的 `pythonpath` 与 `testpaths`，并设置了 ruff 的目标 Python 版本、行长度和检查规则。这些配置使质量检查可以在项目目录中稳定执行。

A terminal screenshot showing the execution of pytest and ruff. pytest runs successfully, collecting 2 items and passing them. Then, ruff format and ruff check are run, both passing. The output shows the platform, python version, rootdir, configfile, testpaths, collected items, and the final result of 2 passed in 0.01s.

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ ruff format --check . && ruff check
4 files already formatted
All checks passed!
```

图 2: 第 13 题 pytest、ruff format 与 ruff check 通过

截图显示，pytest 共收集两个测试项目，最终结果为“2 passed”。随后 `ruff format`

--check . 显示全部文件格式正确，ruff check 显示 “All checks passed!”，说明测试、格式与静态检查均通过。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ nano check.sh
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ chmod +x check.sh
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.00s =====
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ echo $?
0
```

图 3: 第 13 题 check.sh 执行结果与退出码

截图显示，check.sh 成功执行了格式检查、静态检查和测试，最后 echo \$? 输出为 0。返回值 0 表明该质量门禁脚本执行成功，可在提交前作为统一检查入口使用。

2.2 第 14 题让 Make 只重建真正受影响的产物

题目原文

在 q14 中按给定内容创建三个源文件，只需完成 Makefile。

给定内容如下：

```
# data.csv
name,value
alpha,2
beta,3
gamma,5

# stats.py
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))

# build_report.py
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")

# report.md
```

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

实验步骤

本题先创建数据文件 data.csv、统计脚本 stats.py、报告生成脚本 build_report.py 以及报告标题文件 report.md。其中，stats.py 用于计算 CSV 文件中数值列的总和，build_report.py 用于读取统计结果并生成最终报告。

随后编写 Makefile，明确目标文件之间的依赖关系。其核心内容如下：

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: stats.txt report.md build_report.py
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
```

通过首次执行 make、再次执行 make、执行 touch data.csv 后重新执行 make，可以验证 Make 的增量构建机制。最后使用 make clean 删除构建产生的中间文件与结果文件。

实验结果与分析

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano data.csv
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano stats.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano build_report.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano report.md
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano Makefile
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make
python3 stats.py
python3 build_report.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ cat stats.txt
10
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ cat report.txt
# 第4周数据报告

Total: 10
```

图 4: 第 14 题首次 make 构建及生成结果

首次执行 make 时,终端依次运行 python3 stats.py 和 python3 build_report.py。生成的 stats.txt 内容为 10, report.txt 中包含报告标题和 “Total: 10”, 说明数据统计与报告生成均成功完成。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make
make: Nothing to be done for 'all'.
```

图 5: 第 14 题第二次 make 未重复构建

第二次直接运行 make 时,终端显示 “Nothing to be done for ‘all.’”。这表示源文件没有发生变化,目标文件仍然是最新状态,因此 Make 没有重复执行构建命令。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ touch data.csv
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make
python3 stats.py
python3 build_report.py
```

图 6: 第 14 题更新 data.csv 后重新构建

执行 touch data.csv 后再次运行 make,终端重新执行 stats.py 与 build_report.py。这是因为数据文件的时间戳更新后, stats.txt 以及依赖它的 report.txt 都需要重新生成。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make clean
rm -f stats.txt report.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ find . -maxdepth 1 -type f -printf '%f\n' | sort
Makefile
build_report.py
data.csv
report.md
stats.py
```

图 7: 第 14 题 make clean 清理结果

执行 `make clean` 后，终端删除 `stats.txt` 和 `report.txt`。随后列出的目录文件中仅保留 `Makefile`、源数据文件、`Python` 脚本和 `Markdown` 文件，说明 `clean` 目标只清理生成物，没有删除源文件。

2.3 第 15 题把本地 API 数据转换为可读报告

题目原文

在 `q15` 中创建给定 `JSON` 和本地 `HTTP` 服务，再用 `curl` 与 `jq` 生成简短 `Markdown` 报告。

给定 `JSON` 如下：

```
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

- 把给定 `JSON` 保存为 `packages.json`，并在 `q15` 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。
- 使用 `jq` 筛选 `status` 为 `active` 且 `downloads` 不少于 100 的记录，并按 `downloads` 降序、`name` 升序排列。
- 编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 `name/version/downloads` 三列表格。

实验步骤

首先将题目给定的 `JSON` 数据保存为 `packages.json`，并在 `q15` 目录启动本地 `HTTP` 服务。随后使用 `curl -fsS` 获取本地服务器返回的 `JSON` 数据，再使用 `jq` 完成筛选、排序和格式化。

脚本 `api_report.sh` 先输出 `Markdown` 标题与表头，再对 `JSON` 数组执行筛选和排序，最终生成 `summary.md`。筛选逻辑的关键部分如下：

```
map(select(.status == "active" and .downloads >= 100))
| sort_by(-.downloads, .name)
```

```
| .[]  
| "| \(.name) | \(.version) | \(.downloads) |"
```

筛选后排除 inactive 的 beta 和 downloads 小于 100 的 epsilon。delta 和 gamma 的下载量同为 450，因此按照名称升序排列，最终顺序为 delta、gamma、alpha。

实验结果与分析

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ curl -fsS http://127.0.0.1:8000/packages.json | jq .  
[  
  {  
    "name": "alpha",  
    "status": "active",  
    "downloads": 120,  
    "version": "1.2.0"  
  },  
  {  
    "name": "beta",  
    "status": "inactive",  
    "downloads": 900,  
    "version": "2.0.0"  
  },  
  {  
    "name": "gamma",  
    "status": "active",  
    "downloads": 450,  
    "version": "1.5.1"  
  },  
  {  
    "name": "delta",  
    "status": "active",  
    "downloads": 450,  
    "version": "0.9.0"  
  },  
  {  
    "name": "epsilon",  
    "status": "active",  
    "downloads": 80,  
    "version": "3.1.0"  
  }  
]
```

图 8: 第 15 题 curl 获取本地 JSON 数据

截图显示，`curl -fsS http://127.0.0.1:8000/packages.json` 能够成功访问本地 HTTP 服务，`jq` 将 JSON 数据格式化输出。该结果说明本地 API 服务和数据文件均可正常访问。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ sed -n '1,220p' api_report.sh  
#!/usr/bin/env bash  
set -euo pipefail  
  
{  
  printf '# 本地 API 软件包报告\n\n'  
  printf '| name | version | downloads |\n'  
  printf '| --- | --- | ---: |\n'  
  
  curl -fsS http://127.0.0.1:8000/packages.json |  
    jq -r '  
    map(select(.status == "active" and .downloads >= 100))  
    | sort_by(-.downloads, .name)  
    | .[]  
    | "| \(.name) | \(.version) | \(.downloads) |"  
    ,  
  } > summary.md
```

图 9: 第 15 题 api_report.sh 脚本内容

截图显示，脚本使用 `set -euo pipefail` 保证命令执行出现错误时立即停止。脚本通过 `curl` 获取 JSON，再使用 `jq` 筛选 `active` 状态且 `downloads` 不少于 100 的记录，并将结果写入 `summary.md`。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ ./api_report.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ cat summary.md
# 本地 API 软件包报告

| name | version | downloads |
| --- | --- | ---: |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 10: 第 15 题 `summary.md` 报告结果

生成的 Markdown 报告包含 `name`、`version`、`downloads` 三列，最终保留 `delta`、`gamma` 和 `alpha` 三条记录。结果中的排序符合下载量降序、名称升序的要求。

2.4 第 16 题修复并交付一个陌生的小型工具仓库

题目原文

复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查。

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 `Hello, name!`，运行 `check.sh` 确认测试失败；随后定位并修复。
- 编写最小 Makefile，包含 `check`、`build` 和 `clean`；`check` 调用 `check.sh`，`build` 生成 `wheel`。
- 运行 `make check` 和 `make build`，计算 `wheel` 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。

实验步骤

本题首先将 q13 复制为 q16，并继续使用仓库根目录已有的 Git 管理。随后按题目要求，将问候语实现临时改为固定输出 `Hello, name!`，运行 `check.sh` 以复现测试失败。

在确认失败后，将输出逻辑修复为根据参数中的姓名输出问候语：

```
print(f"Hello, {name}!")
```

修复完成后重新执行 `check.sh`，确认代码质量检查和两个测试用例均通过。

之后编写最小 Makefile，将检查、构建与清理任务封装为 `make` 目标：

```
.PHONY: check build clean

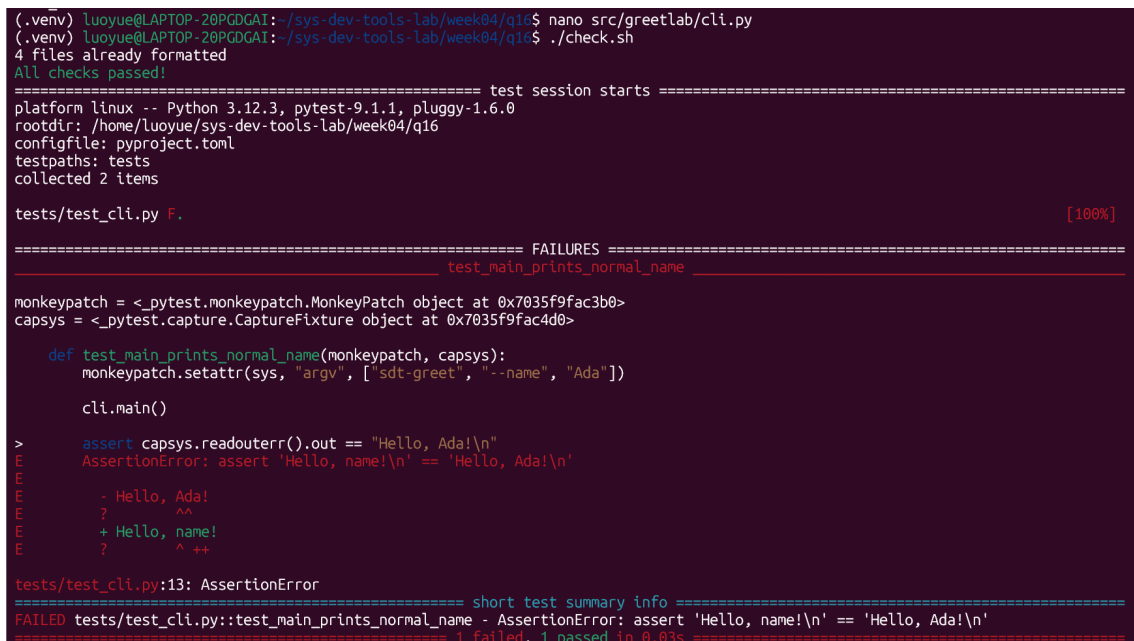
check:
    ./check.sh

build:
    python -m build

clean:
    rm -rf build dist src/*.egg-info
```

最后运行 `make build` 构建源码包和 wheel 包，并通过 `sha256sum dist/*.whl` 计算 wheel 文件的 SHA-256 校验值。

实验结果与分析



```
(.venv) luoyue@LAPTOP-20PCDGA1: ~/sys-dev-tools-lab/week04/q16$ nano src/greetlab/cli.py
(.venv) luoyue@LAPTOP-20PCDGA1: ~/sys-dev-tools-lab/week04/q16$ ./check.sh
4 files already formatted
All checks passed!

===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/Luoyue/sys-dev-tools-lab/week04/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py F. [100%]

===== FAILURES =====
test_main_prints_normal_name

monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x7035f9fac3b0>
capsys = <pytest.capture.CaptureFixture object at 0x7035f9fac4d0>

    def test_main_prints_normal_name(monkeypatch, capsys):
        monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Ada"])
        cli.main()

> assert capsys.readouterr().out == "Hello, Ada!\n"
AssertionError: assert 'Hello, name!\n' == 'Hello, Ada!\n'
- Hello, Ada!
?      ^^
+ Hello, name!
?      ^++

tests/test_cli.py:13: AssertionError

===== short test summary info =====
FAILED tests/test_cli.py::test_main_prints_normal_name - AssertionError: assert 'Hello, name!\n' == 'Hello, Ada!\n'
1 failed, 1 passed in 0.03s
```

图 11: 第 16 题故意修改问候语后测试失败

截图显示，将问候语临时修改为固定文本 `Hello, name!` 后，正常姓名测试失败。测试期望输出为 `Hello, Ada!`，但实际输出为 `Hello, name!`，因此触发断言错误。该结果说明已有测试能够有效发现功能回归。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.00s =====
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ echo "exit_code=$?"
exit_code=0
```

图 12: 第 16 题修复后 check.sh 通过

恢复为基于姓名变量输出问候语后，执行 `check.sh` 的结果显示两个测试全部通过。终端输出 `exit_code=0`，说明格式检查、静态检查和测试均执行成功。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ cat Makefile
.PHONY: check build clean

check:
    ./check.sh

build:
    python -m build

clean:
    rm -rf build dist src/*.egg-info
```

图 13: 第 16 题 Makefile 内容

截图显示，Makefile 中定义了 `check`、`build` 和 `clean` 三个目标。其中 `check` 调用 `check.sh`，`build` 调用 `python -m build`，`clean` 用于删除构建产生的 `build`、`dist` 和 `egg-info` 文件夹。

```
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/__init__.py -> build/bdist.linux-x86_64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.linux-x86_64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_24160001055.egg-info to build/bdist.linux-x86_64/wheel/./greetlab_24160001055-0.1.0-py3.12.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab_24160001055-0.1.0.dist-info/WHEEL
creating '/home/luoyue/sys-dev-tools-lab/week04/q16/dist/.tmp-xj5pme2j/greetlab_24160001055-0.1.0-py3-none-any.whl' and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab_24160001055-0.1.0.dist-info/METADATA'
adding 'greetlab_24160001055-0.1.0.dist-info/WHEEL'
adding 'greetlab_24160001055-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_24160001055-0.1.0.dist-info/top_level.txt'
adding 'greetlab_24160001055-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_24160001055-0.1.0.tar.gz and greetlab_24160001055-0.1.0-py3-none-any.whl
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ sha256sum dist/*.whl
c7b76ade6741dba0694ede24d558d5e6cc6dd89ccef73c16f142701318446f8a dist/greetlab_24160001055-0.1.0-py3-none-any.whl
```

图 14: 第 16 题 make build 构建过程

截图显示，`make build` 成功调用 `python -m build`，并完成源码包构建流程。构建过程中出现 README 文件缺失的警告，但没有导致构建中断，后续仍成功生成了发布产物。

```
(.venv) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/q1e$ make build
python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
running egg_info
creating src/greetlab_24160001055.egg-info
writing src/greetlab_24160001055.egg-info/PKG-INFO
writing dependency links to src/greetlab_24160001055.egg-info/dependency_links.txt
writing entry points to src/greetlab_24160001055.egg-info/entry_points.txt
writing top-level names to src/greetlab_24160001055.egg-info/top_level.txt
writing manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
reading manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
* Installed build dependency versions:
  - setuptools==84.0.0
* Building sdist...
running sdist
running egg_info
writing src/greetlab_24160001055.egg-info/PKG-INFO
writing dependency links to src/greetlab_24160001055.egg-info/dependency_links.txt
writing entry points to src/greetlab_24160001055.egg-info/entry_points.txt
writing top-level names to src/greetlab_24160001055.egg-info/top_level.txt
reading manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
warning: sdist: standard file not found: should have one of README, README.rst, README.txt, README.md

running check
creating greetlab_24160001055-0.1.0
creating greetlab_24160001055-0.1.0/src/greetlab
creating greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
creating greetlab_24160001055-0.1.0/tests
copying files to greetlab_24160001055-0.1.0...
copying pyproject.toml -> greetlab_24160001055-0.1.0
copying src/greetlab/_init_.py -> greetlab_24160001055-0.1.0/src/greetlab
copying src/greetlab/cli.py -> greetlab_24160001055-0.1.0/src/greetlab
copying src/greetlab_24160001055.egg-info/PKG-INFO -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/SOURCES.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/dependency_links.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/entry_points.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/top_level.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
```

图 15: 第 16 题 wheel 构建完成并计算 SHA-256

截图显示,构建过程最终成功生成 .tar.gz 与 .whl 文件。随后使用 `sha256sum dist/*.whl` 输出 wheel 文件的 SHA-256 校验值。该值可用于验证构建产物的完整性,便于后续提交与交付时进行核验。

3 实验总结与感悟

本周实验围绕代码质量、增量构建、数据处理与软件交付展开。第 13 题通过 `pytest`、`ruff` 和 `check.sh` 建立了可重复执行的本地质量门禁,使代码格式、静态检查和功能测试能够由统一命令完成。第 14 题通过 `Makefile` 声明文件之间的依赖关系,验证了 `Make` 能够根据源文件变化只重建必要产物,从而提高构建效率。

第 15 题将 JSON 文件通过本地 HTTP 服务提供,再结合 `curl` 与 `jq` 完成数据获取、筛选、排序和 Markdown 报告生成,体现了命令行工具在数据处理任务中的组合能力。第 16 题则将错误复现、测试失败、代码修复、质量验证、构建 wheel 和计算 SHA-256 校验值串联起来,形成了较完整的软件交付流程。

通过本周实验,进一步理解了自动化检查、构建脚本和校验机制在软件开发中的作用。代码不仅应当能够运行,还应当具备明确的测试依据、可重复的构建过程和可验证的交付产物。